Build a Basic CI Pipeline

Assignment: CI Pipeline for a Node.js Project

Scenario

You are a junior developer on a team. Your team lead has asked you to set up a basic Continuous Integration pipeline for a new Node.js project. The pipeline must check out the code, speed up future runs by caching dependencies, save the test results as a downloadable file, and automatically create a GitHub Issue if the tests fail.

Learning Objectives

By the end of this assignment, you will be able to:

1. Understand what a GitHub-hosted runner is, and what happens when you use one that does not exist.
2. Navigate the GitHub Marketplace to find trusted actions.
3. Understand how to use the actions/checkout action.
4. Implement dependency caching using actions/cache.
5. Save workflow files using actions/upload−artifact.
6. Use actions/github−script to interact with the GitHub API conditionally.

Prerequisites

- A free GitHub account.
- Basic knowledge of Git (commit and push).
- No prior coding experience is needed; all code is provided.

1. Part 1: Project Setup

1. Create a new **public** repository on GitHub named ci−pipeline−assignment.
2. Clone the repository to your local machine.
3. Create a file named package.json in the root folder and paste this exact code:

```
{
  "name": "ci-pipeline-assignment",
  "version": "1.0.0",
  "scripts": {
```

```
    "test": "echo 'All tests passed successfully!' && exit 0"
  }
}
```

4. Commit and push this file to your main branch.

2. Part 2: Warm-Up — Break It On Purpose

Before you build the real pipeline, you will run a workflow with a **wrong runner** on purpose. The goal is to see *how* GitHub reacts and where the error shows up. This will make you comfortable reading failed runs later.

2.1 Step 1: Create a deliberately broken workflow

Create a new file at .github/workflows/warmup.yml and paste this exact code:

```
name: Warm-Up (Broken Runner)

on:
  push:
    branches: [main]

jobs:
  broken:
    runs-on: windows-1999

    steps:
      - name: Say hello
        run: echo "This will never run."
```

Notice the line runs−on: windows−1999. That is **not** a real runner. GitHub-hosted runners look like ubuntu−latest, windows−latest, or macos−latest. A made-up name like windows−1999 should be rejected by GitHub.

2.2 Step 2: Push and watch it fail

```
git add .github/workflows/warmup.yml
git commit -m "Add broken warm-up workflow"
git push
```

Open your repository on GitHub and click the **Actions** tab.

2.3 Step 3: Answer these questions

Look at the failed run. Write your answers down (you will submit them):

1. Did the job start at all? Or did it fail before any step ran?

2. What error message does GitHub show? Copy it exactly.

3. Which part of the YAML file caused the problem — the on:, the jobs:, the runs–on:, or the steps:?

4. What is a **GitHub-hosted runner**? Read the official page and write one sentence in your own words: GitHub-hosted runners.

5. What would have happened if you had written runs–on: ubuntu–latest instead?

2.4 Step 4: Fix it

Now change runs–on: windows–1999 to runs–on: ubuntu–latest, commit, and push again.

```
git add .github/workflows/warmup.yml
git commit -m "Fix runner to ubuntu-latest"
git push
```

Go back to the **Actions** tab. The new run should now pass with a green checkmark.

What you should have learned: The runs–on: key chooses the machine that runs your job. GitHub-hosted runners are virtual machines GitHub creates for you on demand. If you ask for a runner that does not exist, GitHub fails the job immediately — none of your steps ever run. You can read the full list of supported runner labels in the GitHub-hosted runners documentation.

You can delete warmup.yml now (optional), or keep it as a reminder of what a broken runner looks like.

3. Part 3: The Real Tasks

Create a new file at .github/workflows/ci.yml.

Start with this skeleton. It already has the **trigger** (runs on push and pull_request to main), the **job** (build), the correct **runner** (ubuntu–latest), and the first step (checkout). You will extend it in the five tasks below.

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4
```

```
      # Task 2: Add caching step here
      # Task 3: Add install, test, and upload-artifact steps here
      # Task 4: Add "create issue on failure" step at the bottom
      # Task 5: Add "generate and upload image" step here
```

Part	What it does
on: push / pull_request	Trigger — the workflow runs when code is pushed to main or a PR targets main.
jobs: build	The job name. A workflow can have one or many jobs.
runs–on: ubuntu–latest	The runner — a fresh Linux VM that GitHub spins up for the job.
steps:	The list of steps that run top to bottom. Each step either uses an action or runs a shell command.

You will build the workflow in five tasks below.

3.1 Task 1: Checkout Code

Goal: Ensure the workflow has access to your repository's code.

What to do: Add a step that uses actions/checkout@v4.

Marketplace Link (if needed): https://github.com/marketplace/actions/checkout

3.2 Task 2: Cache Dependencies

Goal: Speed up the workflow by remembering Node dependencies between runs.

What to do:

1. Go to the Marketplace and search for cache.
2. Find the action created by GitHub (actions/cache).
3. Add a step to your workflow that caches the ~/.npm directory.
4. *Hint: You will need to use a key based on the runner OS and the package.json file.*

Marketplace Link (if needed): https://github.com/marketplace/actions/cache

3.3 Task 3: Upload Test Results

Goal: Save the output of the tests so they can be downloaded after the workflow finishes.

What to do:

1. Add a step that runs the tests and saves the output to a file. Use this exact run command: npm install && npm test > test–output.txt
2. Go to the Marketplace and search for upload artifact .

3. Find the action created by GitHub (actions/upload−artifact).
4. Add a step to upload the test−output.txt file. Name the artifact test−results.

Marketplace Link (if needed): https://github.com/marketplace/actions/upload-a-build-art

3.4 Task 4: Create an Issue on Failure

Goal: Automatically create a GitHub Issue if the tests fail (using the if: failure() condition).

What to do:

1. Go to the Marketplace and search for github−script.
2. Find the action created by GitHub (actions/github−script).
3. Add a step at the very bottom of your workflow.
4. Set the condition to if: failure().
5. Write a script that creates an issue.
6. *Hint: You will need to add permissions: issues: write at the top of your job for this to work. Use the following script:*

```
        script: |
          const runUrl = `https://github.com/${context.repo.owner}/${
context.repo.repo}/actions/runs/${context.runId}`;
          await github.rest.issues.create({
            owner: context.repo.owner,
            repo: context.repo.repo,
            title: ` Workflow failed on ${context.ref.replace('refs/heads
/', '')}`,
            body: `The workflow **${context.workflow}** failed.\n\n- **
Commit:** ${context.sha}\n- **Branch:** ${context.ref.replace('refs/
heads/', '')}\n- **Run:** [View logs](${runUrl})`
          });
```

Marketplace Link (if needed): https://github.com/marketplace/actions/github-script

3.5 Task 5: Upload an Image Artifact

Goal: Generate a simple image on the runner and upload it as an artifact, so you can see how the runner's preinstalled tools work.

What to do:

1. Add a step that installs imagemagick (a tool for creating and editing images). Use this run command: sudo apt−get update && sudo apt−get install −y imagemagick
2. Add a step that uses imagemagick to create a simple PNG image. Use this exact run command: convert −size 200x100 xc:white −gravity center −pointsize 24 −annotate 0 "CI Pipeline

3. Go to the Marketplace and search for upload artifact.
4. Add a step to upload the ci-image.png file. Name the artifact ci-image.

Hint: The ubuntu-latest runner may already have ImageMagick preinstalled, but installing it explicitly ensures the step works on any runner. You can check what tools are preinstalled on GitHub-hosted runners in the actions/runner-images repository.

Marketplace Link (if needed): https://github.com/marketplace/actions/upload-a-build-art

4. Part 4: Verification & Grading Rubric

To complete this assignment, follow these verification steps.

1. Warm-Up Run (Broken Runner)

- Push warmup.yml with runs-on: windows-1999.
- **Check:** The run appears in the **Actions** tab and fails *immediately*.
- **Check:** You wrote down the exact error message GitHub showed.
- **Check:** After fixing to ubuntu-latest, the run passes.
- **Check:** You answered all five warm-up questions in your own words.

2. Verify the Successful Run (Green Path)

- Commit and push your .github/workflows/ci.yml file.
- Go to the **Actions** tab and click the latest run.
- **Check:** Do all steps have green checkmarks?
- **Check:** An Artifact named test-results exists. Download and open it. Does it say "All tests passed successfully!"?
- **Check:** An Artifact named ci-image exists. Download and open it. Does it show a small white image with "CI Pipeline"?
- **Check:** No new issues were created.

3. Verify the Failure Run (Red Path)

- Edit package.json and change the test script to: "test": "echo 'Tests failed!' && exit 1".
- Commit and push.
- **Check:** The workflow fails on the "Install & Test" step.
- **Check:** The "Create issue on failure" step runs and succeeds.
- **Check:** A new issue titled Workflow failed on main appears in your **Issues** tab.
- **Clean up:** Fix package.json back to exit 0, push, and close the test issue.

What to Submit

Submit a link to your public GitHub repository. Your instructor (or you) will verify the existence of the .github/workflows/ci.yml file and check your commit history to ensure:

- A commit that used a wrong runner (the warm-up).
- A commit that fixed the runner.
- A commit with the successful CI pipeline.
- A commit with the intentional test failure and the created issue.
- A final commit that restored the passing tests.

Also submit your written answers to the five warm-up questions in Part 2, Step 3.